INFORME DE LABORATORIO

S11 - Integración con Cliente Frontend: Angular 22 sobre el Gateway, con Keycloak

Datos del estudiante

Nombre:	Witmer Jhetly Mamani Cahuana
Equipo:	Equipo 01
Sesión:	S11 - Integración con Cliente Frontend: Angular 22 sobre el Gateway, con Keycloak
Rol o aporte realizado:	Actualización de runtime a Node.js v22, configuración de Angular 22, verificación de compilación (npm run build), integración de CORS en Gateway y pruebas de cliente Keycloak.
Link de GitHub:	https://github.com/Witmer17/Angular22.git

1. Introducción y Objetivos

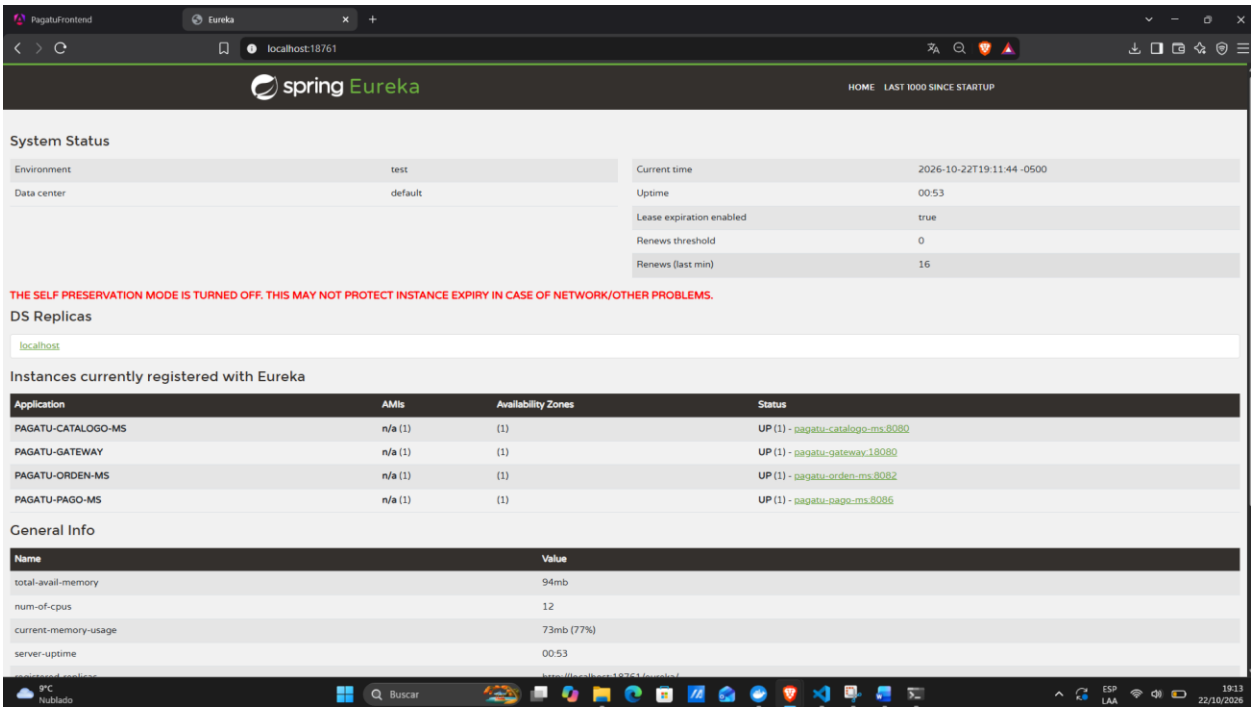
En esta sesión trabajamos en la conexión de la interfaz de usuario con los microservicios que veníamos desarrollando. Hasta las clases anteriores solo probábamos las peticiones usando Postman o directamente entre servicios con OpenFeign, pero ahora necesitábamos que una aplicación web real hecha en Angular 22 pueda consultar los datos y autenticarse sin problemas.

Los objetivos principales que nos propusimos cumplir fueron:

- **Crear y levantar el proyecto en Angular 22** actualizando nuestro entorno a Node.js v22 para que no haya problemas de compatibilidad.
- **Solucionar el problema de CORS** usando el API Gateway como punto único, en lugar de estar abriendo los permisos en cada microservicio por separado.
- **Conectar el login con Keycloak** usando el flujo PKCE para que el usuario pueda iniciar sesión de forma segura desde el navegador.
- **Manejar el token JWT en las peticiones** creando un interceptor en Angular que le pegue el token a cada petición de forma automática.
- **Entender la consistencia eventual en la pantalla** para que el usuario no se confunda cuando una transacción demora unos segundos en procesarse.

2. ¿Cómo está armada la arquitectura?

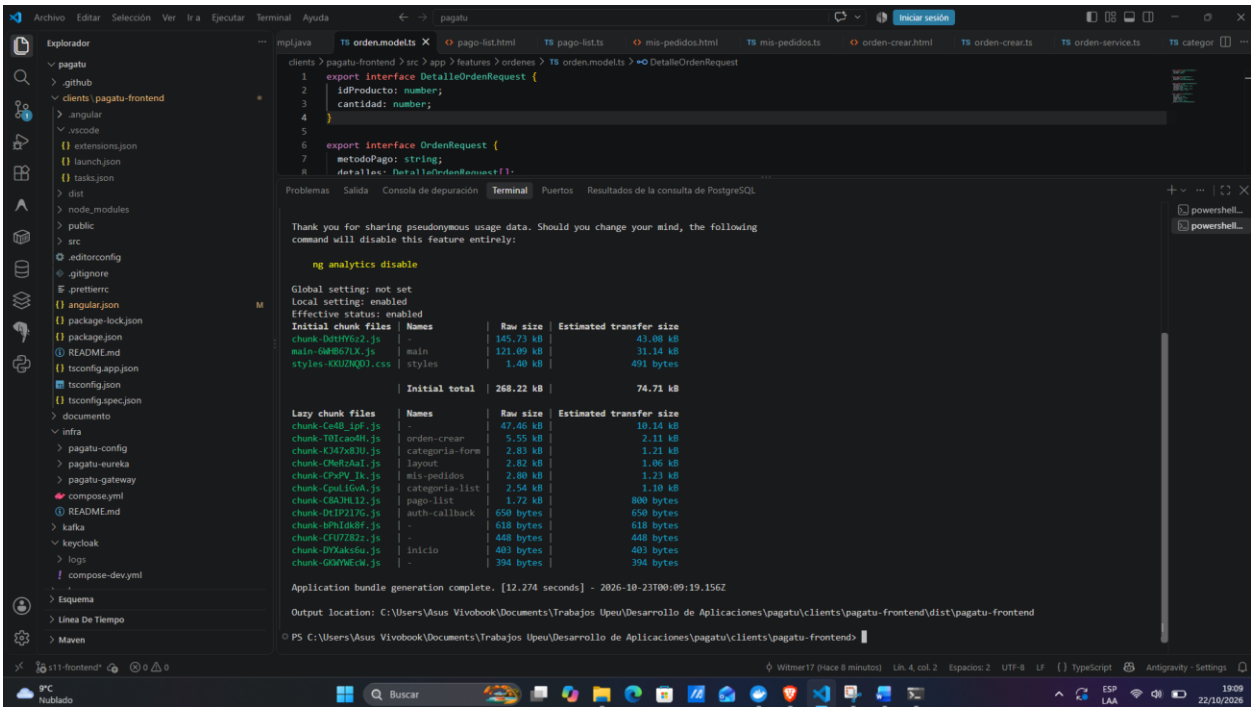
Para que todo funcione de manera ordenada, organizamos la comunicación en tres partes:
• Frontend (Angular 22): Corre localmente en el puerto 4200 y es donde el usuario interactúa.
• API Gateway (Spring Cloud): Es la puerta de entrada. En lugar de que Angular le hable directo a cada microservicio, le hace las consultas al Gateway, y este se encarga de redirigirlas y de responder las cabeceras CORS.
• Keycloak y Microservicios: Keycloak maneja los usuarios, roles y contraseñas. Cuando alguien se loguea, entrega un token JWT que los microservicios validan para saber si la petición es legítima.



3. Desarrollo de la práctica y evidencias paso a paso

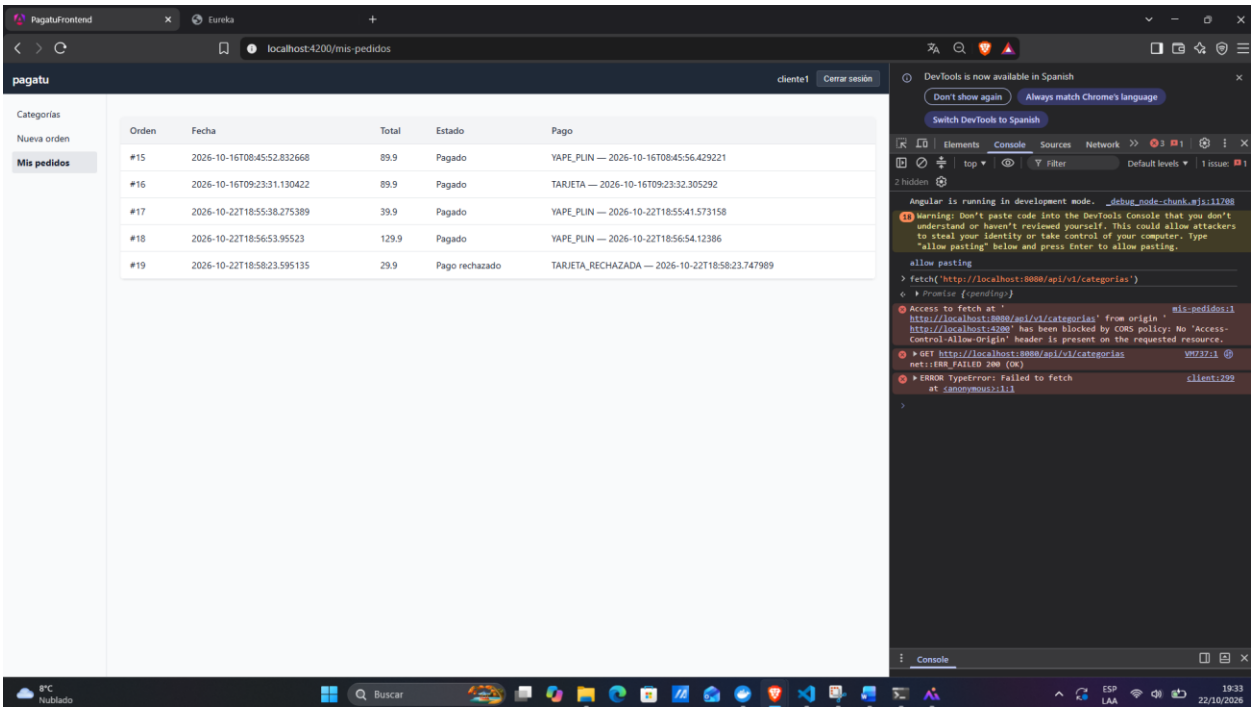
3.1. Preparación del entorno y compilación de Angular

Lo primero que hicimos fue verificar la versión de Node.js, ya que Angular 22 requiere Node 22 para funcionar correctamente. Actualizamos el runtime a la versión v22.23.3 y corrimos el comando de compilación para comprobar que el proyecto base compilara limpio.



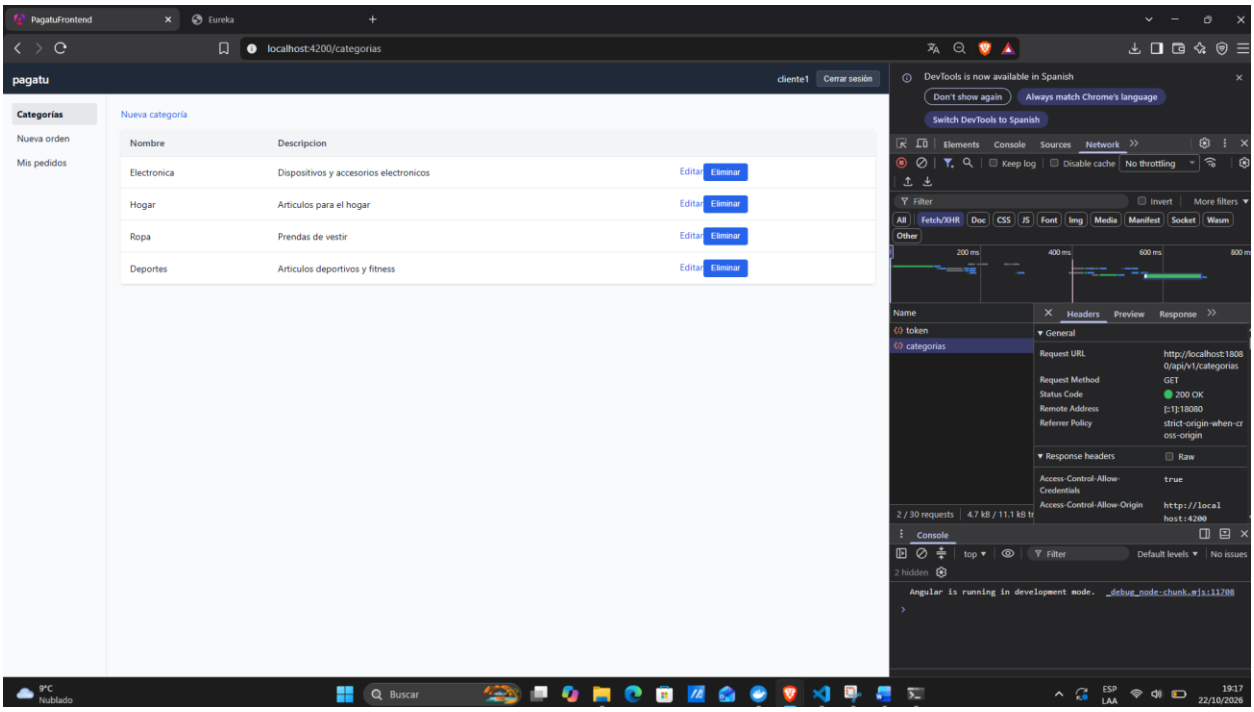
3.2. Parte A: Probando la llamada directa y el error de CORS

Siguiendo la guía, primero intentamos hacer una petición directa desde Angular (localhost:4200) hacia el puerto privado del microservicio sin pasar por el Gateway. Como era de esperarse, el navegador bloqueó la solicitud por la política de mismo origen (CORS). Esto nos sirvió para comprobar en carne propia por qué no se debe llamar directo a los microservicios desde el cliente.



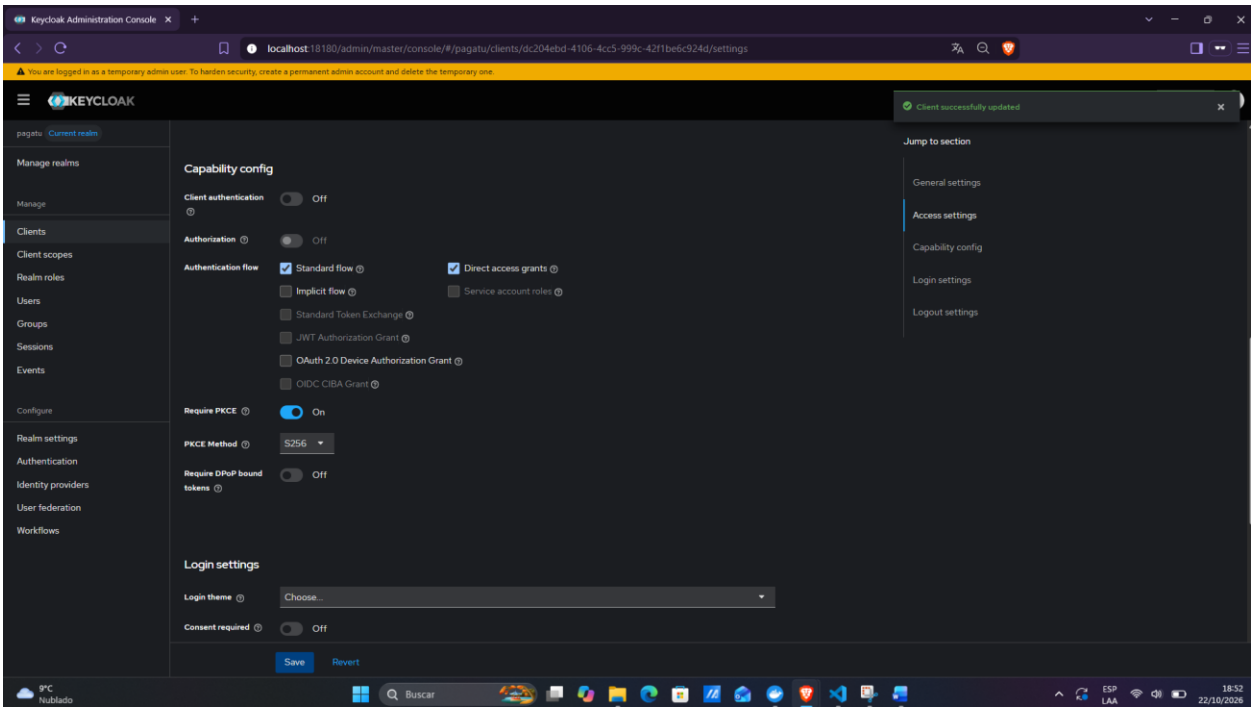
3.3. Parte B: Pasando las peticiones por el Gateway con CORS centralizado

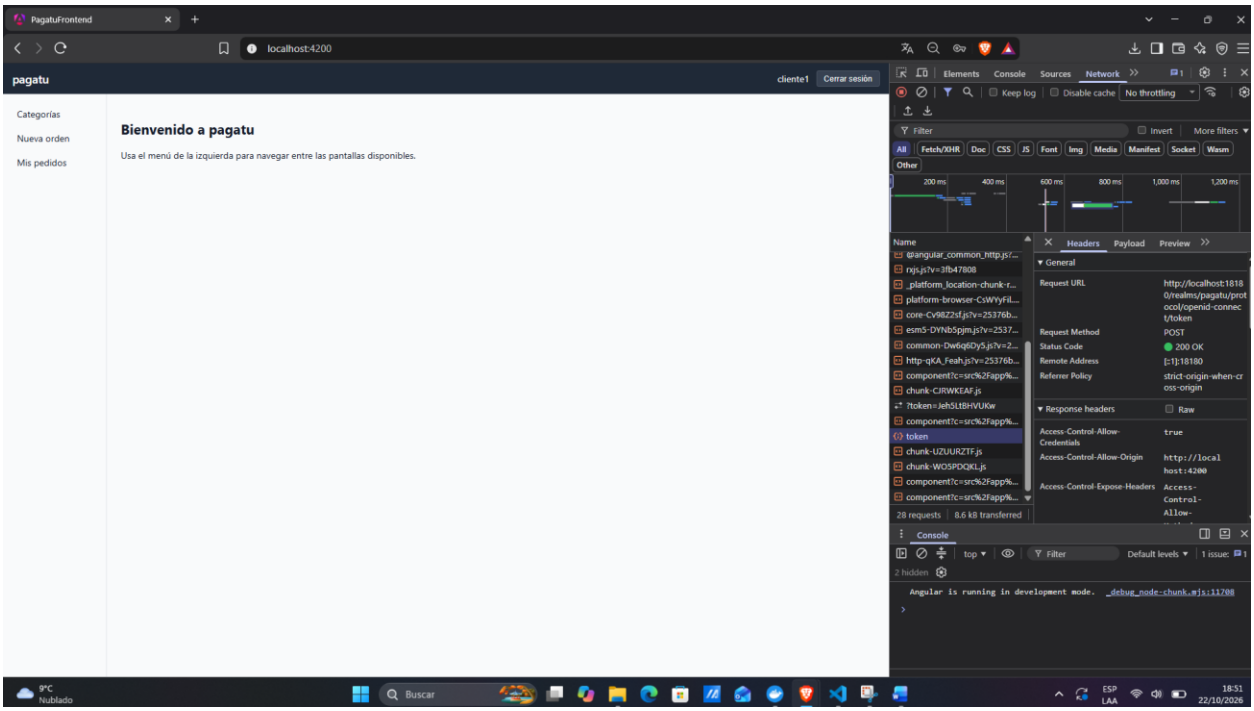
Luego modificamos la URL del servicio en Angular para que apunte al API Gateway en lugar del microservicio directo. En el Gateway configuramos las reglas globales de CORS para permitir solicitudes que vengan de http://localhost:4200. Al hacer esto, la petición respondió con código 200 y los datos cargaron en la pantalla sin que el navegador pusiera ningún reclamo.

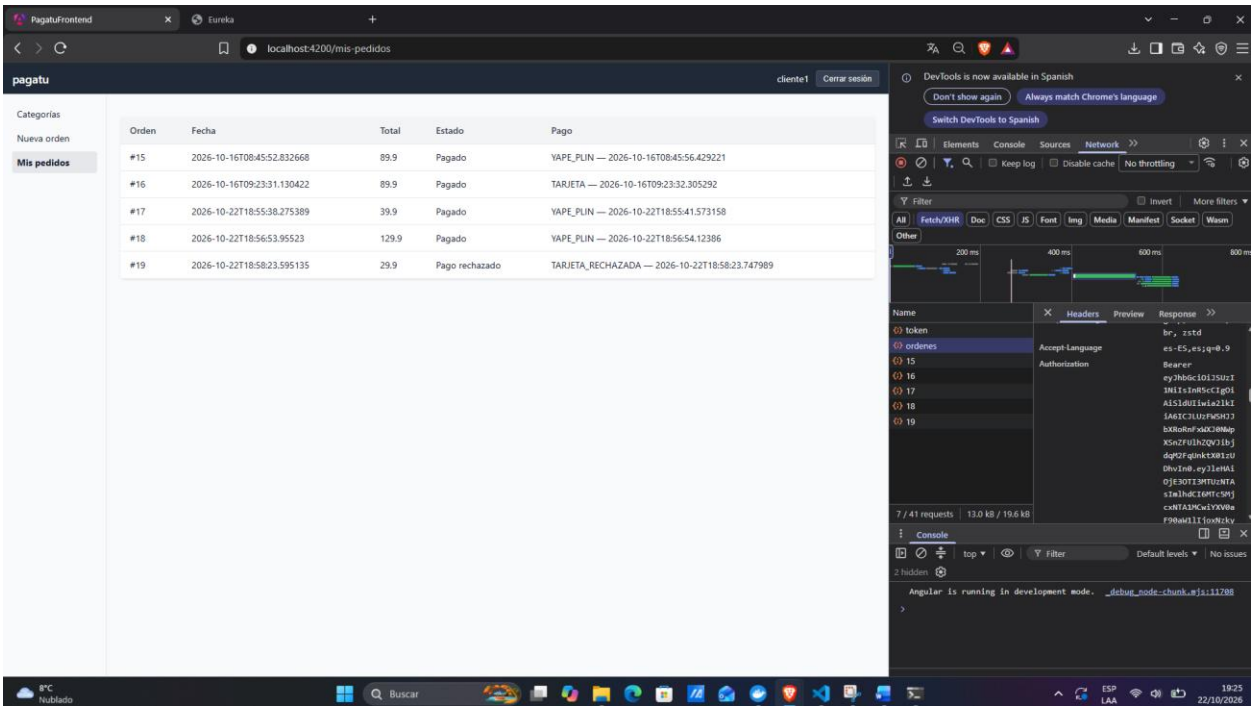


3.4. Parte C: Conexión con Keycloak, Login con PKCE e Interceptor

Para la seguridad, creamos un cliente público en Keycloak llamado 'pagatu-frontend' y le configuramos las URLs de redirección. En Angular usamos la librería de Keycloak con el flujo PKCE. Creamos además un interceptor HTTP para que no tengamos que estar pegando el token manualmente en cada llamada: cada vez que Angular hace un GET o POST, el interceptor le mete la cabecera 'Authorization: Bearer <token>'.

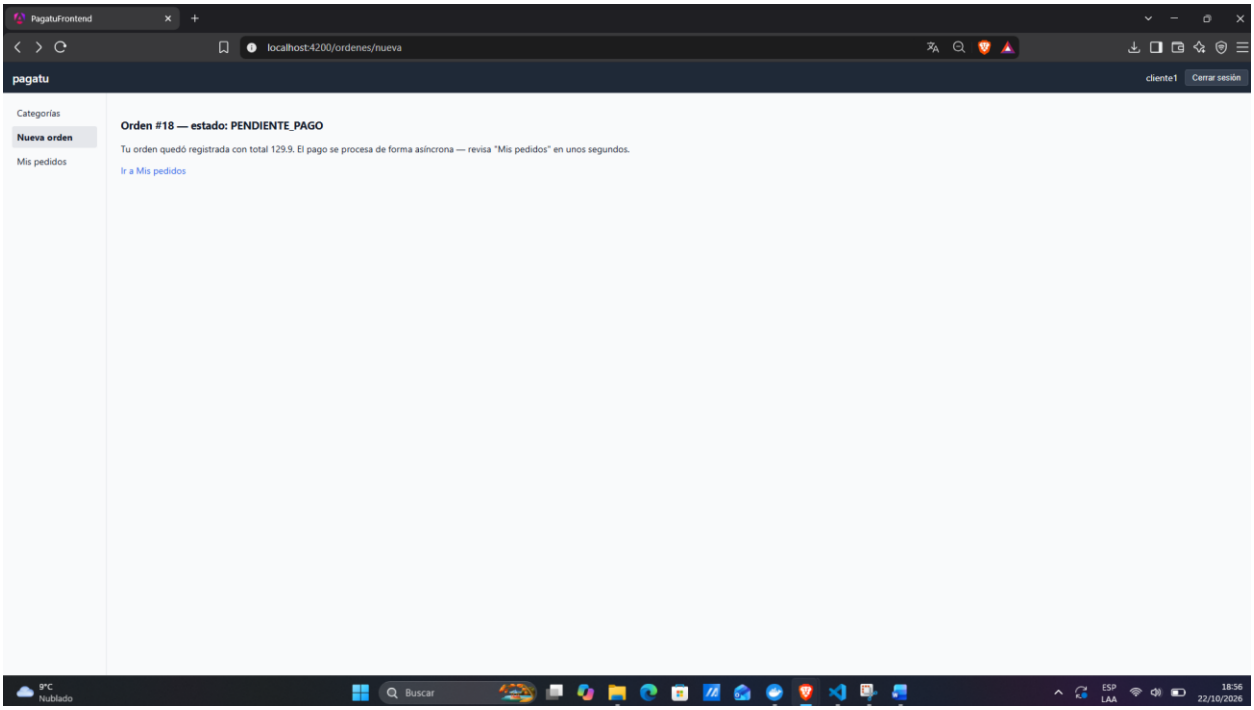

Keycloak Administration Console
localhost:18180/admin/master/console/#/pagatu/clients/dc204ebd-4106-4cc5-999c-42f1be6c924d/settings
You are logged in as a temporary admin user. To harden security, create a permanent admin account and delete the temporary one.
KEYCLOAK
pagatu Current realm
Manage realms
Manage
Clients
Client scopes
Realm roles
Users
Groups
Sessions
Events
Configure
Realm settings
Authentication
Identity providers
User federation
Workflows
Capability config
Client authentication Off
Authorization Off
Authentication flow
Standard flow
Direct access grants
Implicit flow
Service account roles
Standard Token Exchange
JWT Authorization Grant
OAuth 2.0 Device Authorization Grant
OIDC CIBA Grant
Require PKCE On
PKCE Method S256
Require DPoP bound tokens Off
Login settings
Login theme Choose...
Consent required Off
Save
Revert
Client successfully updated
Jump to section
General settings
Access settings
Capability config
Login settings
Logout settings
9°C Nublado
Buscar
ESP LAA
18:52
22/10/2026

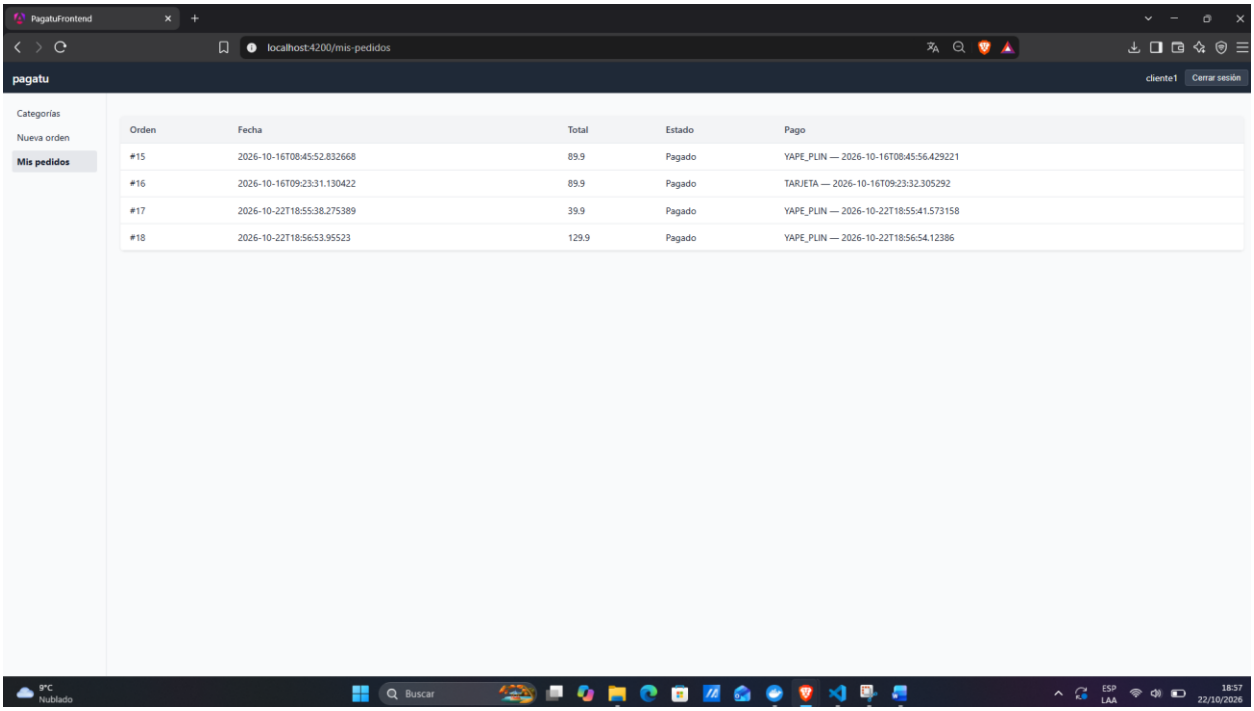

PagatuFrontend
localhost:4200
pagatu
cliente1
Cerrar sesión
Categorías
Nueva orden
Mis pedidos
Bienvenido a pagatu
Usa el menú de la izquierda para navegar entre las pantallas disponibles.
Elements
Console
Sources
Network
Keep log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
200 ms
400 ms
600 ms
800 ms
1,000 ms
1,200 ms
Name
@angular_common_http.js?...
rxjs.js?v=3fb47808
_platform_location-chunk-r...
platform-browser-CsWYyFiL...
core-CV98Z7sf.js?v=25376b...
esm5-DYNb5pjm.js?v=2537...
common-Dw6q6Dy5.js?v=2...
http-qKA_Feah.js?v=25376b...
component?c=src%2Fapp%...
chunk-CJRWKEAF.js
?token=Jeh5LtBHVUKw
component?c=src%2Fapp%...
token
chunk-UZUURZTF.js
chunk-WO5PDQKL.js
component?c=src%2Fapp%...
component?c=src%2Fapp%...
28 requests
8.6 kB transferred
Headers
Payload
Preview
General
Request URL
http://localhost:18180/realms/pagatu/protocol/openid-connect/token
Request Method
POST
Status Code
200 OK
Remote Address
[::1]:18180
Referrer Policy
strict-origin-when-cross-origin
Response headers
Raw
Access-Control-Allow-Credentials
true
Access-Control-Allow-Origin
http://localhost:4200
Access-Control-Expose-Headers
Access-Control-Allow-
Console
top
Filter
Default levels
1 issue:
2 hidden
Angular is running in development mode.
_debug_node-chunk.mjs:11708
9°C Nublado
Buscar
ESP LAA
18:51
22/10/2026

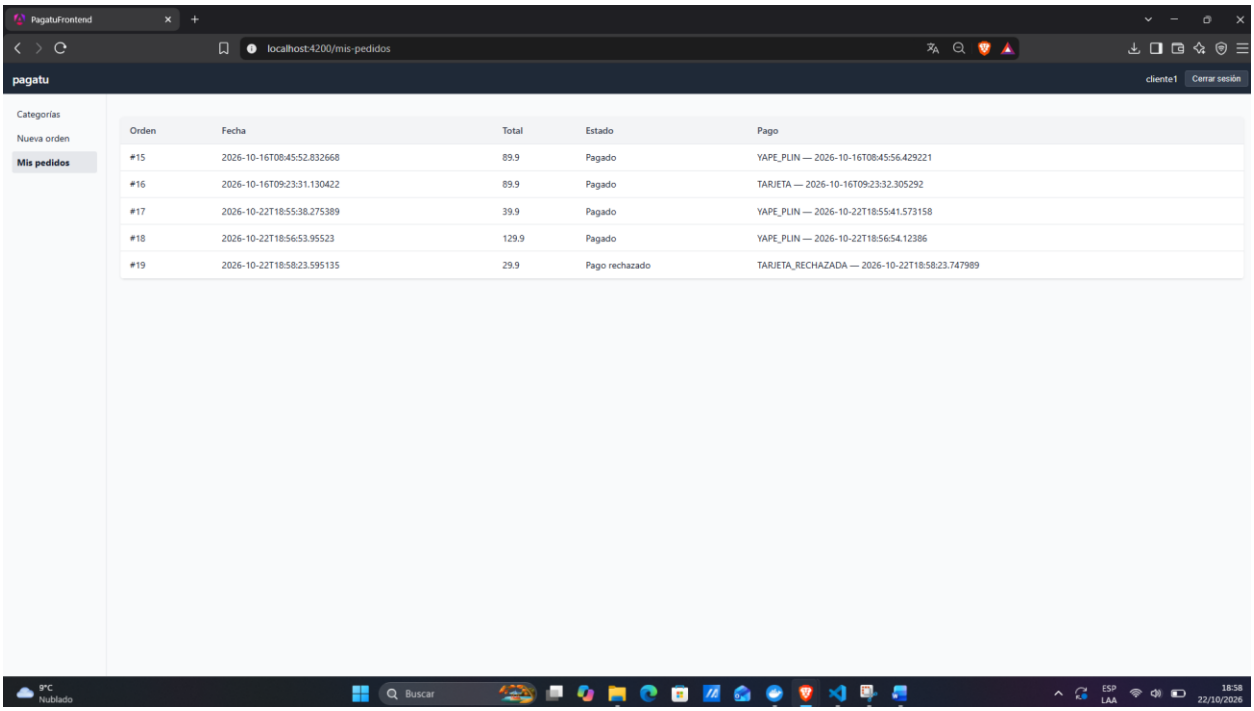


3.5. Parte D: Reflejando la consistencia eventual en la pantalla

Como en el backend usamos mensajería asíncrona (como Kafka), las transacciones no se completan en 1 milisegundo. Por eso armamos una vista donde se muestra primero el estado 'PENDIENTE' y luego, mediante refresco o consulta reactiva, cambia a 'PAGADA' o 'COMPLETADA'. Así el usuario entiende que el sistema está procesando su solicitud y no piensa que la página se colgó.







4. Preguntas de reflexión (Respondidas desde nuestra experiencia)

A continuación respondo las cuatro preguntas de la guía explicando lo que entendí durante la práctica:

1. ¿Por qué en una Single Page Application (SPA) como Angular se usa PKCE y no un Client Secret como en el backend?

Porque una SPA corre directamente en el navegador del usuario. Cualquier persona puede abrir las herramientas de desarrollador (F12) o mirar los archivos JavaScript descargados y encontraría cualquier contraseña o 'Client Secret' que hubiéramos puesto en el código. Es decir, el frontend es un cliente público y no puede guardar secretos.
Con PKCE no se necesita un secret fijo: cada vez que el usuario quiere iniciar sesión, el navegador inventa un código secreto temporal (code_verifier) y manda una versión encriptada (code_challenge). Así, cuando Keycloak devuelve el código de autorización, solo ese mismo navegador puede canjearlo por el token, evitando que alguien lo intercepte en el camino.

2. ¿Qué ventajas tiene configurar CORS solo en el API Gateway en vez de ponerlo en cada microservicio?

La ventaja principal es que te ahorras un dolor de cabeza y mantienes el proyecto ordenado. Si tienes 5 o 10 microservicios y configuras CORS en cada uno, si el día de mañana cambia el puerto o el dominio del frontend tendrías que modificar y redesplegar todos los servicios uno por uno. Además, a veces pasa que el Gateway le agrega una cabecera CORS y el microservicio le agrega otra igual, haciendo que el navegador bote error por encabezados duplicados. Centralizarlo en el Gateway hace que los microservicios solo se preocupen de su lógica y queden bien protegidos en su red interna.

3. ¿Qué pasa si el Access Token vence mientras el usuario está usando la aplicación y cómo lo solucionamos?

Si el token vence (por ejemplo, después de 5 o 15 minutos), la siguiente vez que el usuario haga clic en un botón para consultar datos, el Gateway o el microservicio va a rechazar la petición con un error 401 Unauthorized. Para que esto no sea molesto ni saque al usuario de la aplicación a cada rato, en el frontend se usa el Refresh Token. El interceptor de Angular o la librería de Keycloak se da cuenta de que el token está por vencer y pide uno nuevo a Keycloak por debajo de la mesa (en segundo plano), sin que el usuario tenga que volver a escribir su usuario y contraseña.

4. En un sistema con consistencia eventual, ¿cómo evitamos que el usuario haga doble clic y duplique pagos o compras?

Como los eventos tardan un poquito en procesarse en el backend (por ejemplo mientras Kafka procesa la orden), el usuario puede impacientarse y darle varias veces al botón de pagar pensando que no funcionó. Para evitar esto en la interfaz:
a) Deshabilitamos el botón inmediatamente al primer clic ('disabled') y mostramos un spinner o texto de 'Procesando...'.
b) Le mandamos al backend una clave única (idempotencia) con la petición. Así, si por algún motivo la petición sale dos veces, el backend reconoce el mismo código y no duplica el cobro.
c) Mostramos el estado inicial como 'Pendiente' para que la persona sepa que su orden ya está en cola.

5. Conclusiones personales

- Usar el API Gateway como intermediario simplifica bastante el frontend, ya que en Angular solo nos preocupamos por una sola URL base y el Gateway se encarga de repartir el tráfico y de resolver el problema de CORS.
- El flujo PKCE junto a Keycloak nos da una solución bastante profesional y segura para autenticar a los usuarios en una SPA, sin tener que inventar sistemas de login caseros o inseguros.
- Los interceptores de Angular ahorran muchísimo código repetitivo, porque una vez configurado el interceptor, cualquier servicio HTTP envía automáticamente el token sin tener que escribirlo a mano en cada llamada.
- Aprender a manejar la consistencia eventual en la pantalla es clave: no todo en sistemas distribuidos es instantáneo, y es responsabilidad del frontend darle tranquilidad al usuario mostrando estados claros como 'Pendiente' o 'En proceso'.